1 Add with Carry (4-bit)

```c
#include <stdio.h>

void add(unsigned char A, unsigned char B)
{
   unsigned char sum, carry;
   sum = A + B;
   carry = (A + B) >> 4;

   printf("A = %X\nB = %X\n", A, B);
   printf("Sum = %X\nCarry = %X\n", sum, carry);
}

int main()
{
    unsigned char A, B;

    printf("Enter A and B:\n");
    A = 5; B = 3;
    add(A, B);

    A = 9; B = 6;
    add(A, B);

    return 0;
}
```

2 Data Hazard and Forwarding

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char *op[] = {"LW", "ADD", "SUB", "MUL"};
    char *dest[] = {"R1", "R3", "R5", "R7"};
    char *src[] = {"R2", "R1", "R3", "R5"};

    int stalls = 0, forwards = 0;

    for(int i = 1; i < 4; i++)
    {
        if(strcmp(dest[i-1], src[i]) == 0)
        {
            printf("Hazard between %s and %s\n",
                   op[i-1], op[i]);

            if(strcmp(op[i-1], "LW") == 0)
            {
                printf("STALL inserted\n");
                stalls++;
            }
            else
            {
                printf("FORWARDING applied\n");
                forwards++;
            }
        }
    }
    printf("Total Forwardings: %d\n", forwards);
    printf("Total Stalls: %d\n", stalls);
    return 0;
}
```

3 Instruction Pipeline (7 cycles)

```c
#include <stdio.h>

int main()
{
   char *in[] = {"ADD","SUB","MUL"};
   char *st[] = {"IF","ID","EX","MEM","WB"};
   int c,i,s;

   for(c=1;c<=7;c++)
   {
       printf("\n%d\t",c);
       for(i=0;i<3;i++)
       {
           s=c-i-1;
           if(s>=0&&s<5) printf("%s\t",st[s]);
           else          printf("-\t");
       }
   }

   return 0;
}
```

4 Interrupt-Driven I/O

```c
#include <stdio.h>
#include <unistd.h>

void ISR()
{
    printf("\n*** Interrupt Received ***\n");
    printf("I/O Operation Completed\n");
    printf("Returning to Main Program\n");
}

int main()
{
    int i;
    printf("Interrupt-Driven I/O Simulation\n");

    for(i = 1; i <= 10; i++)
    {
        printf("CPU executing task %d\n", i);
        sleep(1);

        if(i == 5)
            ISR();
    }

    printf("All CPU tasks completed\n");
    return 0;
}
```

5 Memory-Mapped I/O

```c
#include <stdio.h>

#define LED_ADDRESS 0xFF00

unsigned char memory[65536];

void writeMemory(unsigned short address, unsigned char data)
{
    memory[address] = data;
    if (address == LED_ADDRESS)
        printf("LED Status: %s\n", data ? "ON" : "OFF");
}

int main()
{
    printf("Memory-Mapped I/O Simulation\n");

    writeMemory(LED_ADDRESS, 1);
    writeMemory(LED_ADDRESS, 0);
    writeMemory(LED_ADDRESS, 1);

    return 0;
}
```

6 Booth's Multiplication

```c
#include <stdio.h>

int main()
{
    int M, Q, A = 0, Q1 = 0, count = 8;

    printf("Enter multiplicand: ");
    scanf("%d", &M);
    printf("Enter multiplier: ");
    scanf("%d", &Q);

    int Q0, product;

    while (count--)
    {
        Q0 = Q & 1;

        if (Q0 == 1 && Q1 == 0)
            A -= M;
        else if (Q0 == 0 && Q1 == 1)
            A += M;

        Q1 = Q & 1;
        Q = (Q >> 1) | ((A & 1) << 7);
        A >>= 1;
    }

    product = (A << 8) | (Q & 0xFF);
    printf("Product = %d\n", product);

    return 0;
}
```

7 Cache Simulation (FIFO)

```c
#include <stdio.h>

int main()
{
    int n=8, cache[4], blocks[]=[1,2,3,4,5,2,1,3];
    int hits=0, misses=0, front=0, found;

    for(int i=0;i<4;i++) cache[i]=-1;

    for(int i=0;i<n;i++)
    {
        found=0;
        for(int j=0;j<4;j++)
            if(cache[j]==blocks[i]) found=1;
        if(found)
        {
            hits++;
            printf("%d: Hit\n", blocks[i]);
        }
        else
        {
            misses++;
            printf("%d: Miss\n", blocks[i]);
            cache[front]=blocks[i];
            front=(front+1)%4;
        }
    }
    printf("Hits=%d\nMisses=%d\n",hits, misses);
    return 0;
}
```

8 Cache Performance Test

```c
#include <stdio.h>
#include <time.h>
#define SIZE 1000000

int main()
{
    static int a[SIZE];
    long long sum=0;
    clock_t s,e;
    double cache,noCache;

    for(int i=0;i<SIZE;i++) a[i]=i;

    s=clock();
    for(int j=0;j<100;j++)
        for(int i=0;i<SIZE;i++) sum+=a[i];
    e=clock();
    cache=(double)(e-s)/CLOCKS_PER_SEC;

    s=clock();
    for(int j=0;j<100;j++)
        for(int i=0;i<SIZE;i++)
            sum+=a[(i*997)%SIZE];
    e=clock();
    noCache=(double)(e-s)/CLOCKS_PER_SEC;

    printf("Sequential Access: %.3f sec\n", cache);
    printf("Random Access: %.3f sec\n", noCache);

    return 0;
}
```